

Manufacturing Operational Insights Report

Dockerized Entertainment React Service

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Maitry Banduke	EN22CS301571
02	Mohd Quasim	EN22CS301603
03	Mrunali Kamerikar	EN22CS301618
04	Muskan Asija	EN22CS301623

Group Name: G03 D6

Project Number: DO-14

Industry Mentor Name: Vaibhav

University Mentor Name: Akshay Saxena

Academic Year: 2025-2026

Table of Content

1. **Introduction**
 - 1.1 Scope of the Document
 - 1.2 System Overview
2. **System Design**
 - 2.1 Application Design
 - 2.2 Process Flow
 - 2.3 Information Flow
 - 2.4 Components Design
 - 2.5 Key Design Considerations
 - 2.6 API Catalogue
3. **Use Case Diagram**
 - 3.1 Actors
 - 3.2 System Boundary (CI/CD System)
 - 3.3 Step-by-Step Use Case Flow
 - 3.4 Relationships
4. **Interfaces**
 - 4.1 User Interface
 - 4.2 Command Line Interface (CLI)
5. **State and Session Management**
6. **Caching**
 - 6.1 Docker Layer Caching
 - 6.2 Dependency Caching (GitHub Actions)
 - 6.3 Container Image Caching
 - 6.4 Terraform Plugin Caching
7. **Functional Requirements**
 - 7.1 User Access & Authentication
 - 7.2 Application & Configuration Management
 - 7.3 CI/CD Pipeline Automation
 - 7.4 Containerization (Docker)
 - 7.5 Deployment & Orchestration (Kubernetes)
 - 7.6 Container Registry Management
 - 7.7 Monitoring & Workflow Tracking
 - 7.8 CI/CD Strategy
8. **Non-Functional Requirements**
 - 8.1 Security Aspects
 - 8.2 Performance Aspects
9. **References**

1. Introduction

1.1 Scope of the document

This document provides a detailed explanation of the complete lifecycle of the Dockerized Entertainment React Service, focusing on how modern DevOps practices are used to build, manage, and deploy the application efficiently. It explains how the React-based frontend application is developed and containerized using Docker to ensure consistent performance across development, testing, and production environments without compatibility issues, while also covering the implementation of Continuous Integration and Continuous Deployment (CI/CD) pipelines using tools like Jenkins or GitHub Actions that automate tasks such as code integration, testing, building, and deployment, thereby reducing manual effort and minimizing human errors, and ensuring that every code update triggers an automatic, fast, and reliable delivery process.

Furthermore, the scope includes the use of Kubernetes for container orchestration. It explains how Kubernetes manages the deployment of containers, handles scaling based on demand, ensures load balancing, and provides self-healing capabilities by automatically restarting failed containers. This helps maintain high availability and reliability of the application.

In addition, the document highlights the use of version control systems like Git and GitHub for managing source code and enabling collaboration among team members. It also outlines the overall deployment workflow, starting from code development to live production deployment, giving a clear understanding of each stage involved.

1.2 System Overview

The system is designed to automate the complete deployment process of the Dockerized Entertainment React application using modern DevOps tools such as Docker, Kubernetes, and CI/CD platforms like Jenkins or GitHub Actions. It creates a fully automated pipeline that starts when a developer pushes code to the GitHub repository. This action automatically triggers the CI/CD pipeline, which performs tasks such as code integration, testing, building the React application, and creating a Docker image. The image is then pushed to a container registry, ensuring that the latest version of the application is always ready for deployment. This automation reduces manual effort, minimizes human errors, and ensures faster and more reliable delivery of application updates.

Docker is used to package the application along with its dependencies into lightweight and portable containers, ensuring consistent behavior across development, testing, and production environments. Kubernetes is responsible for managing these containers by handling orchestration, auto-scaling based on demand, load balancing to distribute traffic efficiently, and rolling updates to deploy new versions without downtime.

It also provides self-healing capabilities by automatically restarting failed containers, ensuring high availability of the application.

1.3 System Design

Application Design

The application is designed using a modern container-based architecture where the Entertainment React frontend is packaged into Docker containers. Each container includes all the required dependencies, ensuring that the application runs consistently across different environments. These containers are managed and orchestrated by Kubernetes, which ensures high availability, scalability, and efficient resource utilization. Kubernetes handles tasks such as deploying containers, managing traffic, and maintaining the desired state of the application, making the system reliable and fault-tolerant.

1.3.1 Process Flow

The process starts at the developer's machine, where the application code is written and managed using Git. Once the code is pushed to the GitHub repository, it automatically triggers the CI/CD pipeline using Jenkins or GitHub Actions. The pipeline then performs tasks such as building the application, running tests, and creating a Docker image. This image is pushed to a container registry and then deployed to the Kubernetes cluster.

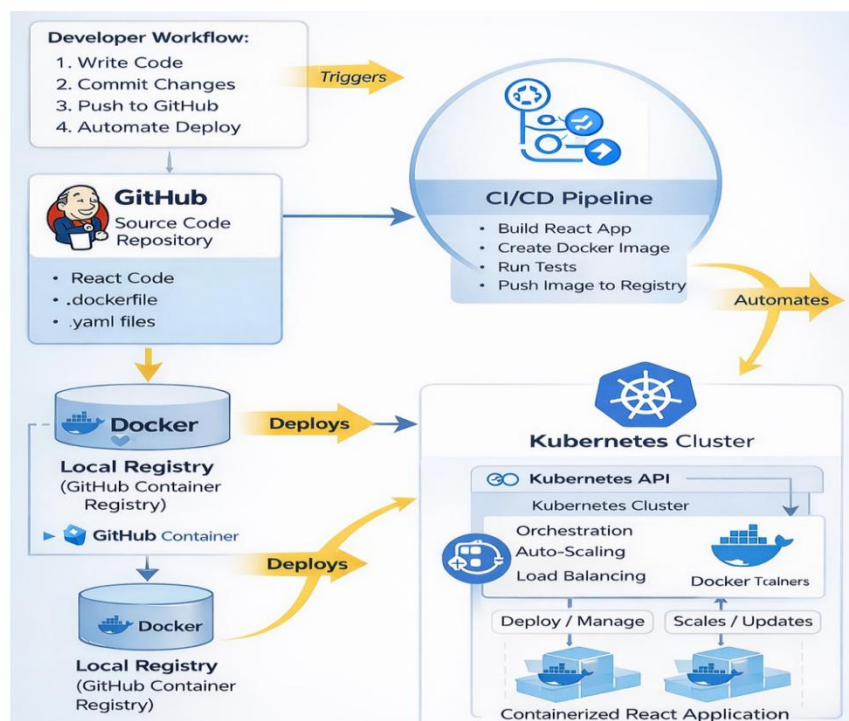


Figure 1: High Level System Architecture

1.3.2 Information Flow

The information flow starts when code is pushed to GitHub, which acts as the central repository. This action automatically triggers the CI/CD pipeline using Jenkins or GitHub Actions. The pipeline then performs steps like building the React application, running tests, and creating a Docker image. This image is pushed to a container registry (Docker Hub or GitHub Container Registry). Finally, Kubernetes pulls the latest image and deploys it to the cluster, ensuring that the updated application is available with minimal manual effort.

1.3.3 Components Design

Different tools work together to complete the deployment process. GitHub is used for version control and code management. Jenkins or GitHub Actions is used to automate the build, test, and deployment process. Docker is used to containerize the React application, making it portable and consistent across environments. Kubernetes is used as the orchestration platform to deploy, manage, and scale the containers efficiently. Each component plays a specific role in ensuring a smooth and automated workflow.

1.3.4 Key Design Considerations

Modularity is maintained by dividing the process into stages such as code management, build, testing, containerization, and deployment, which makes the system easier to manage and update. Scalability is achieved using Kubernetes, which can automatically increase or decrease the number of containers based on user demand. Reliability is also considered, as Kubernetes provides self-healing and rolling updates to ensure the application runs smoothly without downtime.

1.3.5 API Catalogue

The Kubernetes API is used by the CI/CD pipeline to interact with the Kubernetes cluster. It helps in managing tasks such as deploying new versions of the application, scaling the containers, and updating existing services. This allows the entire deployment process to be automated and controlled efficiently without manual intervention.

2. Use Case Diagram for Dockerized entertainment react service

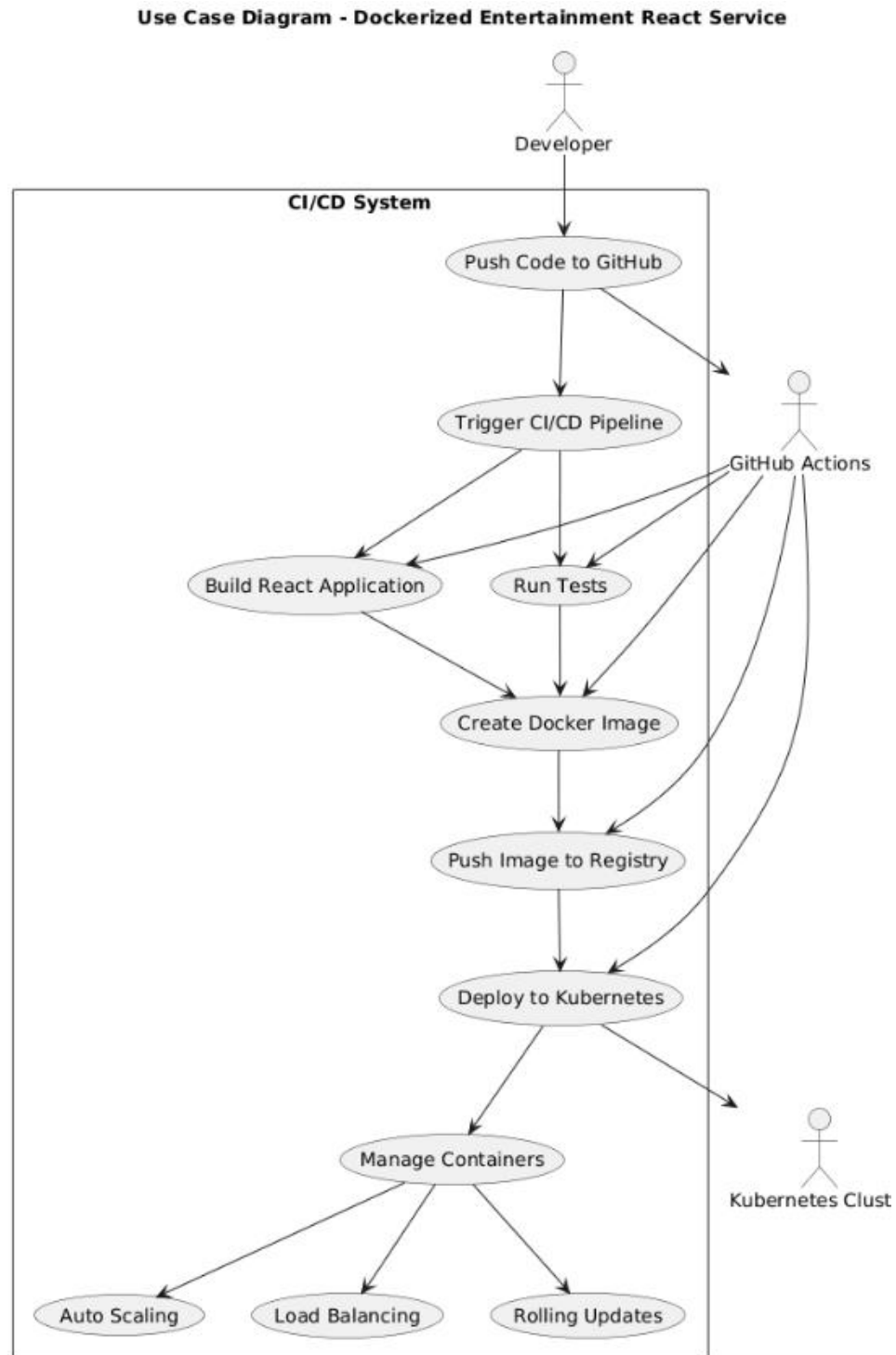


Figure 2: Use Case Diagram

2.1. Actors

- **Developer**
The Developer is the main user of the system. They are responsible for writing code and pushing it to the GitHub repository. This action starts the entire automation process.
- **GitHub Actions (CI/CD System)**
GitHub Actions acts as the automation engine. It handles all CI/CD tasks such as building the application, running tests, creating Docker images, and deploying the application.
- **Kubernetes Cluster**
The Kubernetes Cluster is responsible for running the deployed application. It manages containers and ensures the application is always available and scalable.

2.2 System Boundary (CI/CD System)

The rectangle in the diagram represents the **CI/CD System**, which includes all automated processes from code push to deployment. Everything inside this box is handled automatically without manual intervention.

2.3 Step-by-Step Use Case Flow

Step 1: Push Code to GitHub:

The developer pushes code to the GitHub repository, which starts the system and triggers the automation pipeline.

Step 2: Trigger CI/CD Pipeline

GitHub Actions automatically triggers the CI/CD pipeline after the code is pushed, starting the build and deployment process.

Step 3: Build React Application

The system builds and compiles the React application, ensuring that the code runs correctly without any build errors.

Step 4: Run Tests

Basic tests are executed to verify the functionality of the application and help detect bugs at an early stage.

Step 5: Create Docker Image

A Docker image is created after successful build and testing, packaging the application along with all required dependencies.

Step 6: Push Image to Registry

The Docker image is uploaded to a container registry, making it available for deployment across different environments.

Step 7: Deploy to Kubernetes

Kubernetes pulls the latest Docker image from the registry and automatically deploys the application in the cluster.

2.4 Relationships

- The **Developer** initiates the process.
- **GitHub Actions** is connected to almost all use cases because it performs automation.
- **Kubernetes Cluster** is involved in deployment and container management.
- Arrows show the flow of actions from one step to another.

This use case diagram clearly shows how the system automates the entire lifecycle of the application—from code development to deployment and management. It highlights the role of GitHub Actions in automation and Kubernetes in managing and scaling the application, making the system efficient, reliable, and suitable for modern cloud-based applications.

3. Interfaces

The system provides different interfaces to allow users and developers to interact with the application and deployment process efficiently. These interfaces help in managing code, monitoring the CI/CD pipeline, and controlling the deployment environment. They ensure smooth communication between the user, development tools, and the system.

- **User Interface:**
The GitHub Actions dashboard provides a web-based interface to monitor the entire CI/CD workflow. It allows users to view the status of each pipeline run, including whether the build, test, and deployment stages are successful or failed. Developers can access detailed logs for each step, identify errors, and troubleshoot issues easily. It also shows workflow history, execution time, and real-time progress, making it easier to manage and track deployments.

- **Command Line Interface (CLI):**

The Command Line Interface is mainly used by developers to interact with the system. Git CLI is used for version control operations such as creating branches, committing code, and pushing changes to the GitHub repository, which automatically triggers the CI/CD pipeline. Additionally, Docker CLI can be used to build and test container images locally, and Kubernetes CLI can be used to manage deployments, check pod status, and monitor the application in the cluster.

4. State and Session Management

State and session management in the system ensure that both the application and the deployment pipeline operate reliably and consistently. In the CI/CD process, GitHub Actions maintains the state of each workflow run, including stages such as build, test, and deployment. Each run is isolated and tracked with logs, status reports, and execution history, allowing developers to monitor progress, detect failures, and troubleshoot issues efficiently. This helps maintain transparency and control over the entire deployment lifecycle.

Docker containers are designed to be stateless, meaning they do not store permanent data internally. This makes the system more flexible and scalable, as containers can be created or removed without affecting the overall application. Kubernetes ensures the desired state of the system by continuously monitoring the containers and automatically restarting or replacing any failed instances. It also manages scaling by increasing or decreasing the number of running containers based on demand.

If required, session data can be managed externally using backend services or storage systems, ensuring that user sessions are not lost even if a container restarts. Overall, this approach ensures high availability, fault tolerance, and consistent performance of the application while supporting seamless scaling and reliable user experience.

5. Caching

In this project, caching is used to improve the speed and efficiency of the CI/CD pipeline and deployment process. It helps reduce the time required for building and deploying the Dockerized Entertainment React application by avoiding repeated tasks and reusing previously stored data. This makes the overall system faster, more efficient, and reliable.

- **Docker Layer Caching:**

Docker uses a layered architecture where each step in the Dockerfile creates a separate layer. In this project, when the application is built using GitHub Actions, Docker checks if any changes are made in each step.

If a step remains unchanged (such as installing dependencies), Docker reuses the cached layer instead of rebuilding it. This significantly reduces the build time, as only the updated parts of the application are processed. Earlier, similar caching is also used in tools like Jenkins to speed up building service images, and the same concept is applied here using GitHub Actions.

- **Dependency Caching (GitHub Actions):**

The React application requires various Node.js dependencies. Instead of installing them every time the pipeline runs, GitHub Actions stores these dependencies in a cache. During the next pipeline run, the cached dependencies are reused, which saves time and reduces the need for repeated downloads.

- **Container Image Caching:**

After building the Docker image, it is pushed to a container registry such as Docker Hub or GitHub Container Registry. Kubernetes uses this image for deployment. If the image is already present on the Kubernetes node, it does not need to be downloaded again. This acts as caching and helps in faster deployment and reduced network usage.

- **Terraform Plugin Caching (Conceptual):**

In general DevOps practices, tools like Terraform cache provider plugins locally to reduce deployment time, as they do not need to be downloaded repeatedly. Although Terraform is not directly used in this project, this concept highlights how caching is commonly used in infrastructure automation to improve efficiency and speed.

Overall, caching in this project helps in reducing unnecessary repetition, saving time, and improving the performance of the CI/CD pipeline. It ensures faster builds, quicker deployments, and a smoother development workflow.

6. Functional Requirements

The functional requirements describe in detail how the system should behave and what operations it must perform to successfully automate the development, build, testing, and deployment of the Dockerized Entertainment React application. These requirements ensure that the system is efficient, secure, scalable, and easy to manage using modern DevOps practices and tools like GitHub, Docker, GitHub Actions, and Kubernetes.

6.1 User Access & Authentication

- **GitHub Access Control:** The system uses GitHub as the central platform for managing access. Only authorized team members can push code, create branches, or merge changes into the main repository. This ensures that only verified and approved code is deployed.

- **Repository Security:** Access permissions such as read, write, and admin roles are controlled within GitHub to prevent unauthorized changes.
- **Secure Workflow Execution:** GitHub Actions workflows are triggered only by valid events such as code pushes or pull requests. This ensures that deployments are controlled and cannot be executed by unauthorized users.
- **Branch Protection:** Important branches (like main/master) can be protected to require code reviews before merging, improving code quality and security.

6.2 Application & Configuration Management

- **Environment Variables:** The system must support environment-specific variables such as API endpoints, ports, and configuration settings. These variables allow the application to behave differently in development, testing, and production environments without changing the code.
- **Centralized Code Management:** All application code is stored in GitHub, which acts as the single source of truth. This allows easy tracking of changes, version history, and rollback if needed.
- **Configuration Consistency:** By using environment variables and version control, the system ensures that the same configuration is applied consistently across all environments.
- **Code Collaboration:** Multiple developers can work on the project using branches and pull requests, making collaboration smooth and organized.

6.3 CI/CD Pipeline Automation (GitHub Actions)

- **Automatic Triggering:** The CI/CD pipeline is automatically triggered whenever a developer pushes code to the repository or merges a pull request. This removes the need for manual intervention.
- **Step-by-Step Execution:** The pipeline follows a sequence of steps including installing dependencies, building the React application, running checks, and creating the Docker image.
- **Error Handling:** If any step fails (such as build errors or missing dependencies), the pipeline stops and reports the issue through logs.
- **Parallel Processing:** Some tasks in GitHub Actions can run in parallel, improving speed and efficiency.
- **Reusable Workflows:** The system can use reusable workflow configurations to maintain consistency and reduce duplication in pipeline setup.

6.4 Containerization (Docker)

- **Application Packaging:** Docker is used to package the React application along with all its dependencies into a single container image.
- **Environment Independence:** The Docker container ensures that the application runs the same way regardless of the system it is deployed on.
- **Lightweight & Portable:** Docker images are lightweight and can be easily shared, stored, and deployed across different environments.
- **Versioned Images:** Each Docker image can be tagged with versions, allowing easy tracking and rollback if needed.

6.5 Deployment & Orchestration (Kubernetes)

- **Automated Deployment:** After the Docker image is created, it is automatically deployed to a Kubernetes cluster.
- **Pod Management:** Kubernetes runs the application inside pods and ensures that the required number of pods are always running.
- **Auto-scaling:** Based on traffic or load, Kubernetes can automatically increase or decrease the number of running containers.
- **Load Balancing:** Incoming user requests are distributed evenly across all running containers to ensure efficient performance.

6.6 Container Registry Management

- **Image Storage:** Docker images are stored in a container registry such as Docker Hub or GitHub Container Registry.
- **Version Control for Images:** Multiple versions of images can be stored, allowing rollback to previous stable versions if required.
- **Image Pulling:** Kubernetes pulls the latest or specified image version from the registry for deployment.
- **Secure Access:** Access to the container registry can be controlled using authentication to ensure security.

6.7 Monitoring & Workflow Tracking

- **Pipeline Visibility:** GitHub Actions provides a dashboard where users can monitor each stage of the pipeline.
- **Detailed Logs:** Each step in the workflow generates logs, which help in debugging errors and understanding failures.
- **Execution History:** All workflow runs are stored, allowing users to review past executions and track changes over time.
- **Status Notifications:** Users can quickly identify whether a build or deployment was successful or failed.

6.8 CI/CD Strategy (Continuous Integration & Deployment)

A. Continuous Integration (CI)

- **Automated Build:** Every code change triggers an automatic build process to ensure the application compiles correctly.
- **Error Detection:** The system checks for syntax errors, missing files, or build issues in the React application.
- **Code Validation:** Ensures that the new code integrates properly with the existing codebase.

B. Continuous Deployment (CD)

- **Automated Deployment:** The Docker image is automatically pushed to the container registry and deployed to Kubernetes.
- **Service Continuity:** Kubernetes ensures that the application remains available during updates.
- **Zero Downtime Deployment:** Rolling updates ensure that users do not experience any interruption while new versions are deployed.
- **Fast Delivery:** Updates are delivered quickly to production, reducing deployment time and improving efficiency.

7. Non-Functional Requirements

Non-functional requirements define the quality, performance, and operational standards of the system. They focus on how well the Dockerized Entertainment React Service performs in terms of security, efficiency, reliability, and consistency. These requirements ensure that the system runs smoothly in real-world environments and maintains high standards of performance and protection.

7.1 Security Aspects

Security is an important requirement to protect the application, data, and deployment process from unauthorized access and potential threats.

- **Automated Installation:**
All core dependencies and tools required for the application are installed automatically through scripts and CI/CD workflows using GitHub Actions. This reduces the chances of manual configuration errors, which are one of the common causes of security vulnerabilities. Automated setup ensures that every environment follows the same secure configuration.

- **Secret Masking:**
Sensitive data such as passwords, API keys, and authentication tokens are not stored directly in the source code. Instead, they are managed securely using GitHub Secrets (similar to Jenkins credentials or secret managers). These secrets are encrypted and only accessible during the pipeline execution, ensuring data protection.
- **Access Control:**
Only authorized users are allowed to push code or trigger workflows, ensuring that the system is protected from unauthorized changes.
- **Secure Communication:**
All communication between services such as GitHub, Docker registry, and Kubernetes is done using secure authentication methods.

7.2 Performance Aspects

Performance requirements ensure that the system operates efficiently, responds quickly, and provides a smooth deployment experience.

- **Efficiency:**
Automation through GitHub Actions reduces the time required to set up environments, build applications, and deploy them. Tasks that would normally take hours when done manually are completed in minutes using automated workflows.
- **Consistency:**
The use of Docker containers ensures that the application runs in the same environment across development, testing, and production. This eliminates the issue of “environment drift,” where different systems have different configurations, leading to unexpected errors.
- **Faster Execution:**
By using caching techniques (like Docker layer caching and dependency caching), the system avoids repeating the same steps, which improves speed.
- **Optimized Resource Usage:**
Containers use system resources efficiently, allowing multiple services to run smoothly without performance issues.

These non-functional requirements ensure that the system is secure, fast, and consistent. By reducing manual effort, protecting sensitive data, and improving performance through automation and containerization, the system provides a reliable and efficient environment for deploying the Entertainment React application.

8. References

- Problem Statement: **DO-14 Dockerized Entertainment React Service**
- Project Group: **D6 - Group03**.

9..GitHub- <https://github.com/Mrunali-Kamerikar/Dockerized-Entertainment-React-Service>